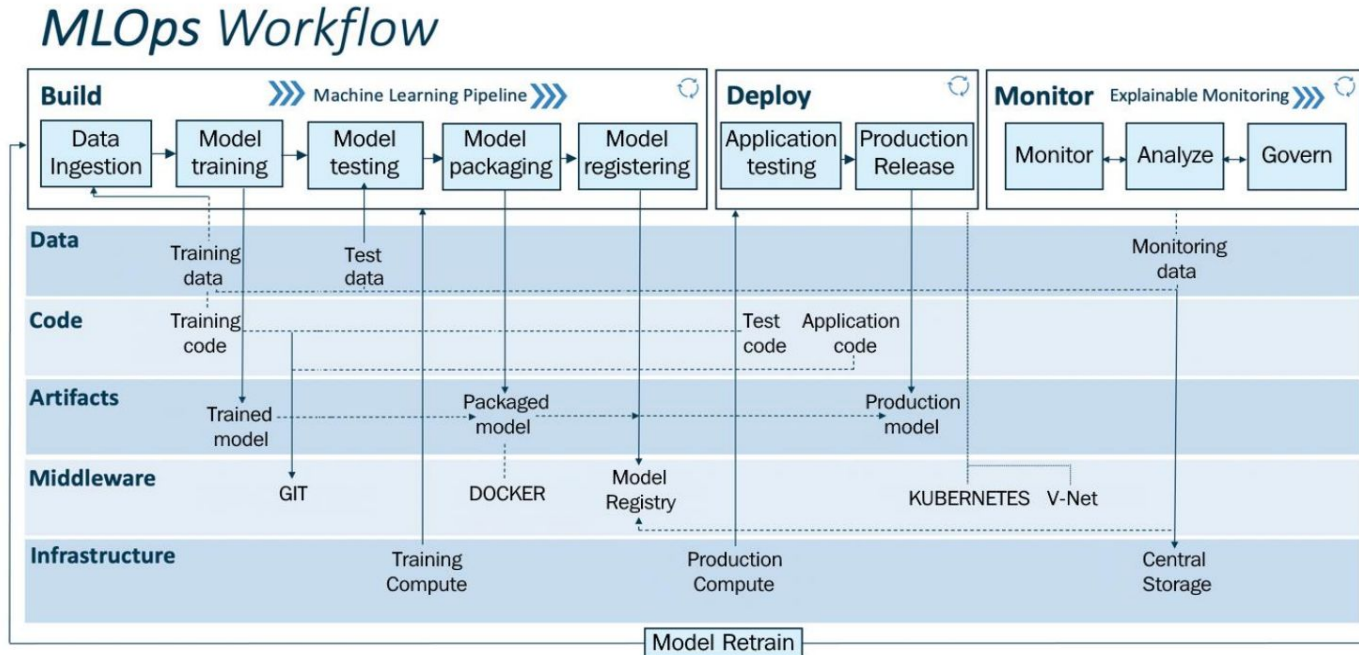


Recap

Alka Bhushan

ML Workflow

Segmented into two modules : (i) MLOps pipeline (ii) Driver



Airflow: Architecture

- **Web Server (UI):** Provides a visual interface to manage and monitor workflows.
- **Scheduler:** Monitors DAGs and triggers tasks based on time or events.
- **Executor:** Manages how tasks are run, distributing them to workers (e.g., SequentialExecutor, CeleryExecutor, KubernetesExecutor).
- **Workers:** Execute the tasks.
- **Metadata Database:** Stores DAGs, task states, and other metadata.

Airflow DAG Design & Workflow Patterns (Recap)

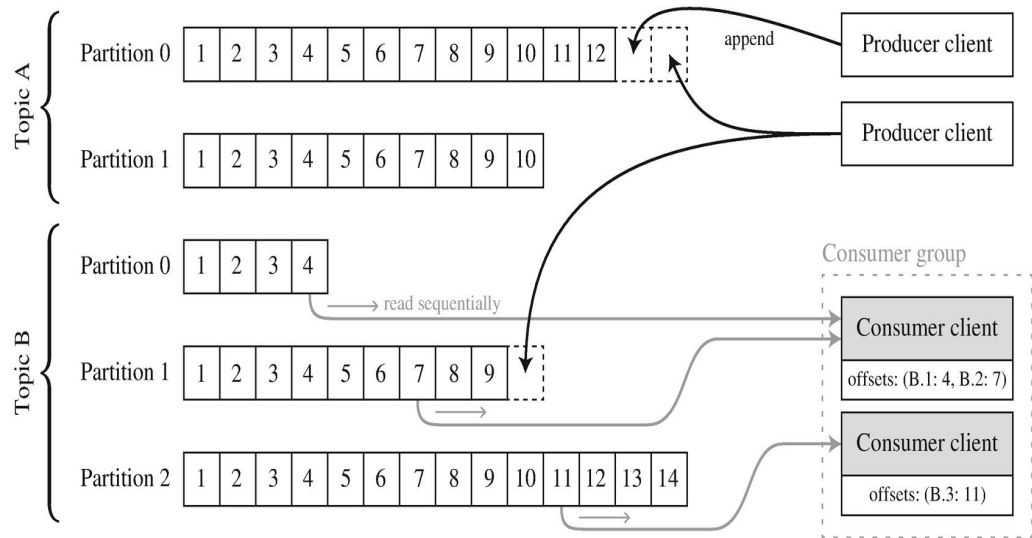
- Core DAG concepts
 - **DAG = Workflow Container** defining tasks, dependencies, schedule, and execution behavior.
 - Scheduling using **Cron Expressions**, **Timedeltas**, or presets such as `@daily`.
 - **Catchup** controls whether missed historical runs are executed.
- Task Creation & Dependencies
 - **BashOperator** for shell commands; **TaskFlow API (@task)** for Python functions.
 - Define task dependencies using `>>` and `<<`.
 - Support for **parallel execution (fan-out/fan-in)**: `t1 >> [t2, t3] >> t4`
- Reliability & Control Flow
 - **on_failure_callback** for alerts and custom error handling.
 - **trigger_rule** controls downstream behavior:
 - `All_success` -> stop on failures.
 - `All_done` -> continue regardless of outcome
- Advanced Workflow Patterns
 - **Sensors** (e.g., `FileSensor`) wait for external dependencies.
 - **Branching** (`@task.branch`) enables conditional execution paths.
 - **Join/Convergence** using `trigger_rule`.

Airflow in Enterprise Systems

- Develop locally, deploy via Git & CI/CD
- Orchestrates workflows, not computation
- Triggered by APIs, applications, or schedules
- Executes Diverse Workloads
 - Data engineering pipelines
 - Big data processing (Spark, Beam, Flink)
 - AI/ML workflows
 - Database and reporting jobs

Apache Kafka - Recap

- Designed for **real-time event streaming and data processing**
- Stores and organizes events using **topics, partitions, and offsets**
- Enables scalable communication through **producers, consumers, and consumer groups**
- Provides **fault tolerance, replication, and high availability**
- Supports **at-most-once, at-least-once, and exactly-once** delivery semantics
- Integrates external systems via **Kafka Connect**
- Enables real-time analytics and transformations using **Kafka Streams**
- Supports horizontal scalability through partitioned topics and distributed brokers
- Widely used in **MLOps** for data ingestion, online inference, monitoring, and retraining



Recap: Apache Spark, Spark ML & Apache Beam

- **Apache Spark** is a distributed computing platform for large-scale **batch processing, streaming, SQL analytics, machine learning, and graph processing**.
- Spark executes applications using **Driver, Cluster Manager, Executors, Jobs, Stages, Tasks, and Partitions** for parallel and scalable processing.
- **Spark ML** provides scalable machine learning capabilities including **feature engineering, ML algorithms, pipelines, model persistence, and hyperparameter tuning**.
- Spark ML workflows are built using **DataFrames, Estimators, Transformers, and Pipelines**.
- **Apache Beam** offers a unified programming model for both **batch and stream processing** that can run on multiple execution engines such as Spark, Flink, and Dataflow.
- Beam introduces concepts such as **Pipelines, PCollections, PTransforms, Windowing, Watermarks, and Triggers** for portable and event-time-aware data processing.

Data Engineering Stack

- Kafka → Data Ingestion & Event Streaming
- Spark → Distributed Batch/Streaming Processing & Analytics
- Beam → Unified Batch and Stream Processing Across Multiple Runners